SUMMATIVE PROJECT

JavaScript Programmer

SkillsTrack Learner Support Portal

A two-month JavaScript, Firebase and GitHub development project

Project Information

Item	Details
Skills Programme	JavaScript Programmer
Skills Programme ID	SP-220373
Curriculum Code	900219-000-00-00
NQF Level / Credits	NQF Level 4 / 60 Credits
Project Duration	2 Months
Assessment	Month 1: 192 marks | Month 2: 190 marks
Total	382 marks
Database	Firebase Realtime Database
Project Method	Team development with individual evidence of competence

Competence Threshold and Academic Integrity

The competence threshold for this assessment is 75%. However, achieving 75% does not automatically mean that a learner will be declared competent.

The assessor must also be satisfied that the learner understands the work submitted and can explain how the solution was planned, developed and applied. A learner may obtain 75% or more but still be declared Not Yet Competent if they cannot explain their code, technical decisions, development process or individual contribution.

Learners must not:

- copy and paste code or answers without understanding them;
- allow one group member to complete all the work;
- submit another person's work as their own; or
- use ChatGPT, Google Gemini or any other artificial intelligence tool to complete the project without understanding and being able to explain the output.

All group members will be assessed and marked individually. Each learner must be able to explain their own contribution and demonstrate an understanding of the full project.

The assessor will ask oral questions before, during or after the assessment to evaluate the learner's understanding, problem-solving ability and application of knowledge.

Any confirmed cheating, plagiarism, unauthorised collaboration or submission of work that is not the learner's own will result in a mark of zero.

Learner name: _Onkarabetse Maluleka, Hilda Lange, Lwandle Sitshi_____

Team name:_____Squad2_____

Assessor: _____Londeka Zikalala_____

Project start date: _____03/08/2026_____ Final due date: ___03/09/20206_____

Month	Marks
Month 1	/192
Month 2	/190
Total	/382

1. Project Purpose and Programme Alignment

The purpose of this project is to provide learners with an integrated opportunity to solve a realistic operational problem using JavaScript and applicable development tools. The finished product must be a functional, readable and testable front-end application supported by Firebase data, REST communication and disciplined version control.

The project addresses the programme requirement to create well-written JavaScript solutions, test and debug code, communicate with web servers, and work collaboratively through Git and GitHub. It combines the knowledge and practical skills developed during the Introduction, Principles, Intermediate Programming and Projects with JavaScript modules.

1.1 Project Assessment Principles

- The project is assessed in two monthly submissions.
- Each learner must submit individual evidence even when the application is developed in a team.
- A feature may receive no marks if the learner cannot explain, demonstrate or modify the submitted code.
- The final mark is calculated out of 382 marks.
- All source code, project documentation and Git history must be available to the assessor.

2. Project Narrative and Client Brief

SkillsTrack Training Centre supports learners who attend short occupational programmes. Learners currently record goals, tasks, support bookings and progress in separate documents and messages. Assessors struggle to see what work is outstanding, which learners need support, and whether learners are making progress.

The centre requires a browser-based Learner Support Portal. The portal must allow registered users to manage learning tasks, book support sessions, review progress, use learning resources and complete a short coding game. The system must use JavaScript for application logic and Firebase as the database.

2.1 Client Objectives

- Provide a single, clear interface for managing learning tasks and support requests.
- Allow users to register, sign in and view their own information.
- Store, retrieve, update and delete application data in Firebase.
- Provide meaningful calculations and summaries based on stored data.
- Include an interactive mini-game that reinforces basic programming concepts.
- Allow a team of developers to collaborate through GitHub and controlled version history.

2.2 Required Users

User	Required access
Learner	Register and sign in; manage own tasks; book support; view progress; play the mini-game; print a progress summary.
Assessor or administrator	View submitted support bookings and relevant learner activity; update booking status where included in the approved scope.

2.3 Minimum Application Features

- User registration, sign-in, sign-out and authenticated user state.
- Dashboard displaying task totals, completed work, outstanding work and calculated progress.
- Task manager with create, read, update and delete functions.
- Support-session booking form with validation and status feedback.
- Search, filter or sort functionality using arrays and higher-order functions.
- Cookie-based preference such as theme, display mode or last selected filter. Passwords and sensitive data may not be stored in cookies.
- Confirmation dialog before a destructive action, a redirect after an appropriate action, and a printable progress summary.
- At least one animation driven by JavaScript timers and one controlled multimedia element.
- A basic operable mini-game created with an Assessor-approved JavaScript framework or library.
- Firebase Realtime Database integration and documented REST API CRUD operations.

3. Required Tools, Technologies and Evidence

Area	Required technology or evidence
Core development	HTML5, CSS3 and JavaScript (ES6 or later).
IDE	Visual Studio Code or another Assessor-approved IDE with formatting, linting and debugging support.
Database	Firebase Realtime Database.
Authentication	Firebase Authentication or another Assessor-approved Firebase authentication method.
REST communication	Firebase Realtime Database REST API using GET, POST, PUT or PATCH, and DELETE. Requests must be authenticated or use an Assessor-controlled test environment.
Framework or library	One Assessor-approved JavaScript framework or library must be used. The game may use Phaser.js, Kaboom.js or another approved option.
Version control	Git and GitHub, including repository, branches, commits, pull requests, merges and contribution history.
Continuous integration	A basic GitHub Actions workflow or equivalent automated check, such as linting or tests.
Testing	Browser developer tools, breakpoints, console, stack traces, manual test cases and evidence of corrections.

3.1 Firebase Data Requirements

The team must design a Firebase data structure before coding. A suggested structure is shown below; teams may adapt it with Assessor approval.

Path	Purpose	Example fields
users/{uid}	Basic learner profile and application role.	displayName, email, role, createdAt
tasks/{taskId}	Learning tasks owned by a user.	userId, title, category, dueDate, priority,

Path	Purpose	Example fields
		completed, createdAt
bookings/{bookingId}	Support-session requests.	userId, topic, preferredDate, notes, status
scores/{scoreId}	Mini-game or quiz results.	userId, score, duration, completedAt
resources/{resourceId}	Optional learning resources.	title, type, url, description

3.2 Security and Data Rules

- Database rules must not permit unrestricted public write access.
- Users should access only data appropriate to their authenticated identity and role.
- Passwords must be handled by Firebase Authentication and may not be stored in the database, cookies or source code.
- Secrets, private credentials and service-account files may not be committed to GitHub.
- User input must be validated before it is written to Firebase.

4. Team Collaboration and Individual Accountability

The project must be completed in teams of three learners unless the Assessor approves another arrangement. Collaboration is assessed, but competence is decided individually.

- Create one shared GitHub repository and a project board or issue list.
- Assign features through issues or tasks and identify the responsible learner.
- Use feature branches rather than making all changes directly on the main branch.
- Open pull requests, conduct at least one peer review per learner, and merge approved work.
- Each learner must make meaningful commits and be able to explain their contribution.
- The team must resolve at least one merge or integration issue and record how it was handled.

4.1 Assessor Review Sessions

- A review must be booked on the provided roster.
- Review appointments are allocated on a first-come, first-served basis.
- Your project must be reviewed by the assessor as per the monthly due dates.
- The team must present a working version, Git history and specific questions.
- Feedback must be recorded, considered and reflected in the following submission.

5. Month 1 – Planning, Tools and Core JavaScript (192 marks)

During Month 1, the team must analyse the problem, plan the solution, configure the development environment, establish the GitHub workflow and create the functional application foundation. The submission must demonstrate understanding of JavaScript fundamentals as well as practical use of variables, functions, arrays, loops, operators, conditionals, DOM events and debugging tools.

5.1 Month 1 Work Schedule

Week	Focus	Expected progress
1	Problem analysis and programming life cycle	Requirements, user stories, flowcharts, pseudocode, scope and testable acceptance criteria.
2	Environment, Git and architecture	IDE configuration, repository, branches, CI check, Firebase data model and class design.
3	Core JavaScript implementation	Variables, functions, arrays, loops, calculations, conditionals, events and dynamic DOM output.
4	Application foundation and review	Interface shell, validation prototype, preferences, debugging evidence, Assessor review and Month 1 submission.

5.2 Month 1 Deliverables

- Problem statement, project scope and client requirements.
- At least six user stories with acceptance criteria.
- Programming life-cycle plan showing analysis, design, coding, testing, implementation review and improvement.
- Flowcharts or pseudocode for login, task creation, progress calculation and deletion confirmation.
- Firebase data model and REST endpoint plan.
- Class or object design showing at least two planned classes and their relationships.
- Configured IDE with formatter, linter and debugging tools.
- Shared GitHub repository with README, .gitignore, issues, branches, pull request evidence and basic CI check.
- Working application shell with navigation and dynamic DOM content.
- JavaScript evidence using variables, data types, operators, functions, scope, arrays, loops and conditionals.
- Cookie preference, dialog or confirmation, redirect and print-function plan or prototype.
- Testing and debugging record with at least three issues and corrections.
- Assessor review record and Month 1 reflection.

5.3 Month 1 Coding Requirements

- Use strict mode in JavaScript modules or scripts where appropriate.
- Use let and const correctly and avoid unnecessary global variables.
- Create reusable functions with parameters and return values, including at least one arrow function.
- Use arrays of data objects and demonstrate map, filter or reduce for a meaningful application result.
- Use loops and conditional statements to process and display data.

- Use event listeners rather than relying only on inline event attributes.
- Create, update or remove at least one DOM element dynamically.
- Apply clear naming, indentation, comments and file organisation.

5.4 Month 1 Formative Questions (92 Marks)

1. Explain why the programming life cycle should be followed before coding begins. ***(2 marks)***

To ensure that the development process is structured and organized, it helps developers understand the problem and plan the solution properly before writing code.

---

2. List and explain the main steps of the programming life cycle. Your answer must show how each step contributes to the development of a working solution. ***(15 marks)***

Problem Definition > clearly identify what the program needs to accomplish. This step ensures that the developers build the correct solution from the beginning.

Program Design > plan the structure and logic of the program before writing actual code. This helps avoid confusion during code and ensures a clear code.

Coding > convent the program design into actual code using a programming language such as Python, java. Coding activities include writing the actual for maintaining the final application .

Testing and Debugging > find errors in the program and ensure it works correctly. This step is crucial for maintenance to ensure it continues to function correctly and efficiently.

Documenting and Ongoing Maintenance > document the program and provide ongoing maintenance to ensure it continues to function correctly and efficiently.

---

3. Explain when const should be used instead of let. Also explain why var should normally be avoided in modern JavaScript. ***(5 marks)***

We use const when the value should not change after creating them and we use let when the value needs to need. Var is a function scoped instead of block scope and it can lead to unexpected behavior, it allows redeclaration of variables and is affected by hoisting making code herder to debug .

---

4. Explain how local and global scope can affect the reliability and maintainability of a JavaScript application. ***(2 marks)***

Local scope improves reliability and maintainability because variables are only accessible within the function or block where they are declared, reducing the risk of accidental modification. Global scope can make applications harder to maintain because global variables can be accessed and changed from anywhere in the program.

---

5. Explain how map(), filter() and reduce() process an array of task objects differently. Provide one suitable use for each method. ***(6 marks)***

map() > creates a new array by applying a function to every element in the original array. It is useful when you want to transform data. Example map() can be used to create an array containing only the task names.

filter() > creates a new array containing only elements that meet a specified condition. It is useful for selecting specific data. For example, filter() can be used to return only tasks that have been completed.

---

reduce() > processes all elements and combines them into a single value. It is useful for calculations and summaries. For example, reduce() can be used to count the total number of completed tasks.

---

6. Explain why an application should use classes or structured objects instead of storing related information in several unrelated variables. ***(5 marks)***

Applications should use classes or structured objects because they group related information together in a single, logical unit. This makes the code easier to understand, manage, and maintain. Objects and classes also reduce duplication and make it easier to update or access related data. Classes and structured objects improve the organisation, reliability, and maintainability of a JavaScript application.

---

7. Explain how branches, pull requests and automated checks reduce risk when developers collaborate on a project. ***(5 marks)***

By improving the quality and safety of code changes. Branches allow developers to work on new features or fixes without affecting the main project. Pull requests enable team members to review code before it is merged helping to identify mistakes and ensure coding standards are followed. Automated checks tests and code validation tools, automatically detect errors, bugs or failed requirements before changes are accepted..

---

8. Study the following values: ***(4 + 1 + 1 marks)***

```
const userName = "Lerato";
const age = 22;
const isActive = true;
const selectedProject = null;
```

a. State the data type of each value. (4 marks)

userName → **String**

age → **Number**

isActive → **Boolean**

selectedProject → **Null**

---

b. Explain how age could be converted from a number to a string. (1 mark)

const ageString(age);

---

c. Explain what the typeof operator is used for. (1 mark)

to check and return the data type of the value

---

9. Analyse the following code: ***(3 + 2 + 2 + 1 marks)***

```
const total = "10" + 5;
const looseComparison = 5 == "5";
```

```
const strictComparison = 5 === "5";
console.log(total);
console.log(looseComparison);
console.log(strictComparison);
```

a. State the output of each console.log() statement. (3 marks)

`console.log(total);= 105`
`console.log(looseComparison);= true`
`console.log(strictComparison);= false`

---

b. Explain why "10" + 5 does not produce the number 15. (2 marks)

Because “10” is a string and 5 is a number.

---

c. Explain the difference between == and ===. (2 marks)

The **==** (loose equality) operator compares values after performing type conversion if necessary.

The **===** (strict equality) operator compares both the value and the data type without type conversion.

---

d. Rewrite the first statement so that it produces the number 15. (1 mark)

`const total = Number ("10") + 5;`

---

10. Study the following functions: ***(2+1+1+1+1+2 marks)***

```
function calculateTotal(price, quantity = 1) {
  return price * quantity;
}

const applyDiscount = (total, percentage) => {
  return total - total * (percentage / 100);
};

const orderTotal = calculateTotal(150, 3);
const finalTotal = applyDiscount(orderTotal, 10);
```

a. Identify the parameters of calculateTotal(). (2 marks)

`function calculateTotal(price, quantity = 1)`

---

b. Explain the purpose of the default value assigned to quantity. (1 mark)

The default value 1 is used if no value is provided for quantity when the function is called.

---

c. Explain what the return keyword does. (1 mark)

The return keyword sends a value back to the place where the function was called and ends the execution of the function.

---

d. State the value stored in orderTotal. (1 mark)

450

---

e. State the value stored in finalTotal. (1 mark)

405

---

f. Explain one difference between a function declaration and an arrow function. (2 marks)

A function declaration is defined using the function keyword and is hoisted, meaning it can be called before it is declared in the code. An arrow function uses the => syntax and is not hoisted in the same way, so it must be defined before it is used.

---

11. Study the following array: ***(2+2+2+2+2 marks)***

```
const tasks = [
  { title: "Create wireframes", completed: true, hours: 3 },
  { title: "Develop login form", completed: false, hours: 5 },
  { title: "Test application", completed: true, hours: 2 },
  { title: "Write documentation", completed: false, hours: 4 }
];
```

Write JavaScript code that:

a. Uses a loop to display the title of every task. (2 marks)

```
for(let i = 0; i<tasks.length; return i++){

console.log(task.title)

}
```

---

b. Uses a conditional statement to display only completed tasks. (2 marks)

```
for (let task of tasks) {
if (task.completed === true) {
console.log(task.title);
}
}
```

---

c. Counts the number of completed tasks. (2 marks)

```
let count = 0;
```

```
 for (let task of tasks) {
 if (task.completed === true) {
  count++;
   }
  }
   console.log(count);
```

---

d. Calculates the total number of hours for all tasks. (2 marks)

```
let totalHours = 0;
 const totalHours = tasks.reduce((sum, task)
return  sum + task.hours,
0);
console.log( totalHours);
```

---

e. Displays an appropriate message if no tasks are available. (2 marks)

```
if (tasks.length === 0) {
console.log("No tasks available.");
} else {
console.log("Tasks are available.");
}
```

---

15. Study the following statement: ***(1+1+1+1 marks)***

```
document.cookie = "theme=dark; max-age=3600; path=/";
```

a. Explain what information the cookie stores. (1 mark)

The cookies stores the user's preference for the website theme

---

b. Explain the purpose of max-age=3600. (1 mark)

Max-age =3600 sets the expiry time of the cookie.

---

c. Explain the purpose of path=/. (1 mark)

The cookies is available to the all pages of the website starting from the root directory.

---

d. Write a JavaScript statement that displays the available cookies. (1 mark)

console.log(document.cookie);

Testing and Problem-Solving

16. A registration form contains the following fields: name; email address; password; and age. Develop five test cases that could be used to test the form. Each test case must include the input or condition being tested and the expected result. Your test cases must include at least one valid submission, one missing value, one invalid email address and one boundary-value test. ***(15 marks)***

Test Case	Input	Expected result
Valid Submission	Name: Onkarabetse Maluleka Email: malulekaonkarabetse@gmail.com Password: Ponkah@22 Age: 20	Form is submitted successfully and registration is completed.
Missing Value	Name: field left blank Email: malulekaonkarabetse@gmail.com Password: Ponkah@22 Age: 20	Form is not submitted. An error message such as "Name is required" is displayed
Invalid email address	Name: Onkarabetse Maluleka Email: malulekaonkarabetse@gmai (missing domain extension) Password: Ponkah@22 Age: 20	Form is not submitted. An error message such as "Invalid email address" is displayed.
Boundary-value test	Name: Onkarabetse Maluleka Email: malulekaonkarabetse@gmail.com Password: Ponkah@22 Age: 17(one below the minimum age)	Form is not submitted. An error message such as "Age must be 18 or older" is displayed.

6. Month 1 Assessment Rubric

Criterion	Assessment standard	Marks
Problem analysis and requirements	Problem, users, scope, constraints, user stories and acceptance criteria are complete and appropriate.	/6
Programming life-cycle plan	Development stages, milestones, iteration and testing activities are planned logically.	/6
Firebase data and REST planning	Data paths, fields, ownership and planned CRUD operations are accurately documented.	/8
Object-oriented design	Classes or structured objects, properties, methods and relationships are planned and justified.	/8
IDE and toolkit configuration	IDE, extensions, formatter, linter, debugger and selected framework or library are configured and demonstrated.	/6
GitHub repository and collaboration setup	Repository, README, .gitignore, issues, branch workflow, pull request and basic CI check are established.	/10
Syntax and disciplined coding style	Code is concise, consistently formatted, meaningfully named, commented and organised.	/6
Variables, data types and operators	Variables, scope, types, conversions and operators are applied correctly.	/8
Functions and scope	Reusable functions, parameters, returns, arrow functions and scope are applied correctly.	/10
Arrays, loops and conditionals	Arrays of objects, array methods, loops and decision structures solve project-relevant problems.	/12
DOM and event foundation	Events and DOM operations produce clear, dynamic and correct application behaviour.	/8
Cookies, dialogs, redirect and print	Safe preference storage and appropriate browser functions are planned or demonstrated.	/4
Testing and debugging evidence	At least three issues are diagnosed with suitable tools, corrected and retested.	/4
Review, reflection and explanation	Feedback is recorded and the learner explains the Month 1 decisions and contribution.	/4
Month 1 summative knowledge question	Summative knowledge question answered and correct.	/92

Total: /192

Comments:

7. Month 2 – Intermediate Development, Firebase and Collaboration (190 marks)

During Month 2, the team must complete the application and integrate authentication, Firebase data, REST requests, object-oriented programming, validation, error handling, debugging, animation, multimedia and a basic JavaScript game. GitHub must show genuine team collaboration and controlled version history.

7.1 Month 2 Work Schedule

Week	Focus	Expected progress
5	Authentication and Firebase	Firebase project, secure rules, authentication, user state and initial database records.
6	CRUD, REST and DOM	Create, read, update and delete operations; dynamic rendering; search, filter and calculated summaries.
7	Intermediate features	OOP implementation, validation, error handling, animation, multimedia and basic game.
8	Integration and assessment	Testing, refactoring, pull requests, final integration, technical review and Month 2 submission.

7.2 Month 2 Deliverables

- Completed and operable Learner Support Portal that meets the approved brief.
- Firebase Authentication with registration, sign-in, sign-out and authenticated user state.
- Firebase Realtime Database with secure rules and structured data.
- Full CRUD for at least one primary entity, such as learning tasks, through Firebase REST requests.
- Evidence of POST, GET, PUT or PATCH, DELETE and a final GET or equivalent verification.
- Object-oriented implementation using ES6 classes, object instances and at least one relationship such as inheritance or composition.
- Client-side validation for names, passwords, email, numeric fields and required data.
- Error handling using try, catch, finally and at least one custom thrown error.
- Debugging and refactoring evidence showing before-and-after code.
- Dynamic DOM operations that create, change and remove elements or attributes.
- JavaScript timer-based animation and controlled image, audio or video content.
- An operable basic game using an Assessor-approved JavaScript framework or library.
- GitHub evidence showing individual branches, commits, pull requests, reviews and merges.
- Final test report, README, contribution report, Assessor review and learner reflection.

7.3 Required Functional Behaviour

Feature	Minimum acceptable behaviour
Authentication	Valid users can register, sign in and sign out. Invalid input produces clear feedback. Authenticated state controls access to personal features.
Task management	Users can create, view, edit, complete and delete tasks. Deletion requires

Feature	Minimum acceptable behaviour
	confirmation.
Progress calculation	The application calculates completed, outstanding and overdue work from task data and displays the result dynamically.
Support booking	Users can submit a validated support request and view an appropriate success or error response.
Search and filters	Users can search, filter or sort tasks or resources using array methods and reusable functions.
Preferences	A non-sensitive preference is written to, read from, modified in and removable from a cookie.
Print and redirect	Users can print a suitable summary and are redirected only when the action and application flow justify it.
Game	The mini-game is playable, records a score or outcome and stores the result or summary in Firebase.

7.4 Month 2 Formative Questions (90 Marks)

1. Explain how the Firebase data model can prevent one user's records from being confused with or displayed as another user's records. Refer to user IDs and document ownership in your answer. ***(6 marks)***

- **Unique User IDs (UIDs)**

  Firebase Authentication assigns each user a unique identifier when they register or log in.

  No two users share the same UID, ensuring distinct identity.

- **Document Ownership via UID Paths**

  In Firestore, data is stored under paths that include the UID

  Example: `users/{uid}/records/{recordId}` keeps each user's data in their own sub- collection.

- **Data Separation**

  Because the UID is part of the path, one user's documents cannot overlap with another's.

  Queries scoped to `users/{uid}` only return that user's records.

- **Security Rules Enforcement**

  Firebase rules check that the authenticated UID matches the UID in the document path.

  This prevents unauthorized access to another user's data.

- **Controlled Access**

  Even if multiple users store similar types of data (e.g., "tasks"), the UID ensures they are isolated in separate collections.

- **Result: Isolation and Safety**

  The combination of unique UIDs, structured document paths, and enforced rules guarantees that each user only sees their own records.

2. n. ***(6 marks)***

Client- Side Validation

Definition: Runs in the browser using JavaScript before data is sent to the server.

Purpose: Ensures inputs meet format and completeness rules (e.g., email looks valid, password length ≥ 8).

Example from the application: When a learner registers, the JavaScript checks that the email contains `@` and that the password meets the minimum length before allowing submission.

Authentication

Definition: Runs on the server (e.g., Firebase Authentication).

Purpose: Confirms the user's identity by verifying credentials against stored records.

Example from the application: When a learner logs in, Firebase Authentication verifies the entered email and password against its database and returns a unique user ID token if correct.

---

3. Explain when POST, GET, PUT or PATCH, and DELETE should be used in a REST workflow. Provide one application-related example for each method. ***(8 marks)***

GET – *Retrieve data*

When to use: To read or fetch information from the server without changing anything.

Application Example: A learner opens their dashboard, and the app sends a GET request to `/users/{uid}/tickets` to display all their support tickets.

POST – *Create new data*

When to use: To send new data to the server, which then creates a new resource.

Application Example: A learner submits a "Help Request" form. The app sends a POST request to `/tickets` with the form details, creating a new support ticket in the database.

PUT / PATCH – *Update existing data*

When to use:

PUT: Replace the entire resource with new data.

PATCH: Modify only specific fields of a resource.

Application Example:

PATCH: A learner updates the status of a ticket from "Open" to "Resolved" without changing other fields.

PUT: A learner edits their profile, and the app sends a PUT request to `/users/{uid}` replacing the whole profile document.

DELETE – *Remove data*

When to use: To permanently delete a resource from the server.

Application Example: A learner deletes one of their uploaded documents with a DELETE request to `/users/{uid}/resources/{docId}`.

---

4. Explain how try, catch, finally, and throw work together when a Firebase or REST request fails. ***(8 marks)***

`try` **block**

Holds the code that *might fail*, such as a Firebase call or REST API request.

Example:

```
js
try {
  const data = await firebase.firestore().getDocs(...);}
```

`catch` **block**

Executes if an error occurs inside the `try`.

Purpose: Handle failures gracefully (e.g., show an error message, retry, or log).

Example:

```
js
catch(error) {
  console.error("Failed to fetch tasks:", error);}
```

`finally` **block**

Runs *no matter what* — whether the request succeeds or fails.

Purpose: Clean up resources, stop loading spinners, or close connections.

Example:

```
js
finally {
  setLoading(false);}
```

`throw` **statement**

Used to deliberately raise an error when something is invalid or unexpected.

Example:

```
js
if (response.status !== 200) {
  throw new Error("Bad response");}
```

Application- Related Example Flow

`try`**:** Attempt to fetch a learner's support tickets from Firebase.

`catch`**:** If the request fails (e.g., network error), display "Unable to load tickets."

`finally`**:** Hide the loading spinner after the request completes, regardless of success or failure.

`throw`**:** If the server returns "Unauthorized," explicitly throw an error so the app forces the user to log in again.

5. A learner's application displays a progress total of 150% instead of 75%. Describe the steps that should be followed to identify and correct the logical error. Refer to debugging tools and the flow of data in your answer. ***(8 marks)***

Reproduce the Problem

Run the application and confirm that the progress bar shows **150% instead of 75%**.

This ensures the bug is consistent and not a one- off glitch.

Trace the Flow of Data

Follow how progress is calculated:

Input values (e.g., completed modules vs total modules).

Formula used to compute percentage.

Check if values are being doubled, miscounted, or incorrectly multiplied.

Inspect with Debugging Tools

Use **console.log()** in JavaScript or Firebase query inspection to print:

Raw data fetched (e.g., `completed = 3`, `total = 4`).

The computed percentage before rendering.

Alternatively, use browser dev tools (Chrome DevTools) to step through the function line by line.

Hypothesize the Cause

Example: The formula might be `completed/total * 200` instead of `completed/total * 100`.

Or the learner's progress is being counted twice due to duplicate entries in Firebase

Apply the Fix

Correct the formula to `progress = (completed / total) * 100`.

Ensure data is deduplicated before calculation.

Retest

Reload the app and confirm the progress now shows **75%**.

Test with different learners to ensure the fix works universally.

---

6. Explain how pull requests and peer reviews can prove individual contribution and improve the quality of a team's code. ***(6 marks)***

Pull Requests

Definition: A formal request to merge code changes from a feature branch into the main branch.

Proves Contribution: Each pull request is linked to the individual's GitHub account, showing exactly what files and lines they changed.

Improves Quality: Team members can review the code before merging, ensuring standards are met and bugs are caught early.

Peer Reviews

Definition: The process where other team members examine the pull request code before approval.

Proves Contribution: Review comments and approvals demonstrate collaboration and highlight who contributed fixes or improvements.

Improves Quality: Encourages multiple perspectives, catches logical errors, enforces coding conventions, and prevents poor practices from entering the main branch.

---

7. Firebase Security and Access Control ***()***

A Firestore database stores learner tasks using the following structure:

```
users
 └── userId
      └── tasks
           └── taskId
```

a. Explain why the authenticated user's ID should be used in the document path. (2 marks)

- **Reason:** Each user has a unique UID generated by Firebase Authentication.
- **Benefit:** Storing tasks under `users/{userId}/tasks/{taskId}` ensures that tasks are isolated per user, preventing overlap or confusion between different learners' data.

---

b. Explain how Firebase Security Rules can restrict access to the correct user. (3 marks)

Rule Setup: Security rules can check that the `request.auth.uid` (the logged-in user's ID) matches the `userId` in the document path.

Effect: Only the authenticated user can read or write their own tasks.

Result: Unauthorized users cannot access or modify another learner's tasks.

---

c. State one risk of allowing unrestricted read and write access to the database. (1 mark)

Risk: Any user could view, change, or delete all learners' tasks, leading to data leaks, tampering, or loss of sensitive information.

---

d. Explain why validation must still be performed even when security rules are present. (2 marks)

- **Reason:** Security rules only control *who* can access data, not *what* data looks like.
- **Benefit:** Validation ensures inputs are correct (e.g., task name not empty, progress between 0–100). This prevents logical errors, malformed data, or abuse even when access is restricted.

---

8. Analyse the following JavaScript error-handling code: ***()***

```
async function saveTask(task) {
  try {
    if (!task.title) {
      throw new Error("A task title is required.");
    }
    const response = await fetch("/tasks", {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify(task)
    });
    const result = await response.json();
    return result;
  } catch (error) {
    console.log("Task saved successfully.");
  } finally {
    console.log("Request completed.");
  }
}
```

a. Explain the purpose of the async keyword. (1 mark)

Declares the function as asynchronous, allowing the use of `await` inside it.

Ensures the function returns a **Promise**, which can resolve or reject later.

---

b. Explain the purpose of await. (1 mark)

Pauses execution until the Promise resolves.

Makes asynchronous code look synchronous, simplifying error handling and flow.

---

c. Explain why a custom error is thrown when the task has no title. (2 marks)

- Prevents invalid data from being sent to the server.
- Provides a clear, descriptive message (`"A task title is required."`) so the developer/user knows exactly what went wrong.

---

d. Identify the incorrect message in the catch block and explain why it is misleading. (2 marks)

The catch block logs **"Task saved successfully."**

This is misleading because the catch block only runs when an error occurs. Instead of success, it should report the actual error

e. Rewrite the catch block so that it reports the actual error. (2 marks)

```
catch (error) {
 console.error("Error saving task:", error.message);
```

This reports the real error instead of falsely claiming success.

f. Explain when the finally block will execute. (2 marks)

- Runs **always**, regardless of success or failure.
- Executes after `try` and `catch` to perform cleanup tasks (e.g., stopping a loading spinner, closing connections).

9. DOM Manipulation — Study the following HTML: ***(8 marks)***

```
<ul id="taskList"></ul>
<button id="addTaskButton">Add Task</button>
```

Explain how JavaScript could be used to create a new <li> element, add task text to it, append it to the list, and later remove it. Name the relevant DOM methods or properties.

- **Create a new** `<li>` **element**
- Use `document.createElement('li')` to generate a new list item node.
- **Add text to the** `<li>`
- Either set `li.textContent = 'New Task'`

  Or create a text node with `document.createTextNode('New Task')` and append it using `appendChild`.
- **Append the** `<li>` **to the** `<ul>`
- First, select the list: `const list = document.getElementById('taskList');`

  Then append: `list.appendChild(li);`
- **Remove the** `<li>` **later**

  Call `li.remove()` directly, or

  Use `list.removeChild(li)` if you want to remove it via the parent.

---

10. Object-Oriented JavaScript — Study the following class: ***()***

```
class Task {
  constructor(title, dueDate) {
    this.title = title;
    this.dueDate = dueDate;
    this.completed = false;
  }
  markComplete() {
    this.completed = true;
  }
}
```

a. Explain the purpose of the Task class. (2 marks)

The `Task` class is a **blueprint for creating task objects**. It models a task with details like its title, due date, and completion status, and provides behavior (methods) to update that status.

---

b. Explain what the constructor does. (2 marks)

The `constructor(title, dueDate)` method runs automatically when a new `Task` object is created. It **initializes the object's properties** (`title`, `dueDate`, `completed`) with the values provided, setting `completed` to `false` by default.

---

c. State the three properties created for every task object. (2 marks)

`title` → the name/description of the task

`dueDate` → when the task is due

`completed` → a boolean showing if the task is finished (`false` initially)

---

d. Explain what happens when markComplete() is called. (1 mark)

The method sets the `completed` property of the task object to `true`, marking the task as finished.

---

e. Write one statement that creates a new Task object. (1 mark)

```
const myTask = new Task("Finish assignment", "2026-10-15");
```

---

11. Explain how JavaScript timers such as setTimeout() and setInterval() differ. Describe one suitable use for each in a web application, and state one consideration when adding animation or multimedia. ***(6 marks)***

Difference between `setTimeout()` and `setInterval()`

`setTimeout(function, delay)` → Executes a function **once** after the specified delay (in milliseconds).

`setInterval(function, delay)` → Executes a function **repeatedly** at the given interval until it is cleared with `clearInterval()`.

Suitable Uses

`setTimeout()` → Useful for **delayed actions**. Example: Show a pop-up message 5 seconds after a user logs in.

`setInterval()` → Useful for **repeated tasks**. Example: Update a clock on a dashboard every second.

Consideration for Animation/Multimedia

When adding animation or multimedia, you must consider **performance and smoothness**.

Using `s`

`etInterval()` for animations can cause **jank** because it doesn't sync with the browser's refresh rate.

Instead, prefer `requestAnimationFrame()` for smooth animations, since it aligns updates with the display's refresh cycle.

Also consider **resource usage**: too many timers can slow down the page or drain battery on mobile devices.

---

12. A learner submits a form that creates a new record in Firebase. The application must then retrieve the record and display it on the page. Describe the complete asynchronous workflow from the moment the user selects the submit button until the new record appears in the interface. Refer to validation, the database request, the response, DOM updates, loading feedback, and error handling. ***(8 marks)***

1. **User Action – Submit Button**

The learner clicks the **submit button**.

JavaScript intercepts the event with an **event listener** (`addEventListener('submit', …)` or on the button click).

2. **Validation (Client-Side)**

JavaScript checks the form inputs before sending:

Required fields not empty.

Correct formats (e.g., email, date).

Logical constraints (e.g., due date not in the past).

If invalid → show error message immediately, prevent request.

3. Database Request (Asynchronous Call)

If valid, JavaScript sends the data to Firebase using an **API call** (`firebase.firestore().collection('tasks').add({...})`).

This is asynchronous, meaning the browser doesn't freeze while waiting.

A **loading indicator** (spinner or "Saving…") is shown to the user.

4. Response Handling

Firebase returns a **Promise** with either:

Success → includes the new record's ID and stored data.

Error → failure message (e.g., network issue, permission denied).

5. DOM Update (Interface)

On success, JavaScript retrieves the new record (`.get()` or listens with `.onSnapshot()`).

A new `<li>` or card element is created (`document.createElement`) and populated with the record's data.

It is appended to the page (`appendChild`), so the learner sees the new task instantly.

6. Loading Feedback Removed

Once the record is displayed, the loading indicator is hidden.

The interface shows confirmation (e.g., "Task added successfully").

7. Error Handling

If the request fails:

The loading indicator is removed.

An error message is displayed (`alert()` or inline text).

Optionally, allow retry without losing entered data.

-

---

8. Month 2 Assessment Rubric

Criterion	Assessment standard	Marks
Functional solution	The completed application solves the stated problem and all major user journeys operate correctly.	/6
Object-oriented implementation	Classes, objects, properties, methods and relationships are implemented appropriately and explained.	/8
Firebase database and CRUD	Data is structured, secure and correctly created, retrieved, updated and deleted in Firebase.	/14
Authentication and validation	Registration, login, authenticated state and input validation operate correctly with useful feedback.	/8
REST API communication	Documented GET, POST, PUT or PATCH and DELETE requests communicate correctly with Firebase or an approved web service.	/10
DOM and event handling	The application dynamically creates, changes and removes interface content through clear event-driven code.	/8
Error handling, debugging and refactoring	Errors are handled using appropriate mechanisms; bugs are diagnosed; code is improved without changing intended behaviour.	/8
Animation and multimedia	JavaScript timers or animation functions and controlled multimedia are purposeful and functional.	/6
Basic framework-based game	An operable JavaScript game uses an approved framework or library and records a meaningful outcome.	/8
Data processing and calculations	Functions, arrays, map/filter/reduce, loops and operators produce accurate project-relevant results.	/6
Code quality, tests and documentation	Code is readable, modular and tested; README and test evidence allow the solution to be reviewed and run.	/6
GitHub teamwork and version control	Each learner demonstrates branches, commits, pull requests, reviews, merges and collaborative problem solving.	/10
Technical demonstration and reflection	The learner demonstrates the solution, answers questions and reflects on contribution and improvements.	/2
Month 2 summative knowledge question	Summative knowledge question answered and correct.	/90

Total: /190

Comments:

13. Academic Integrity and Authenticity Declaration

I declare that:

- the evidence submitted for assessment accurately represents my own contribution;
- I have acknowledged tutorials, libraries, code examples, artificial-intelligence assistance and other external sources;
- I understand the JavaScript, Firebase and GitHub work that I am presenting;
- I can explain, test and modify the code attributed to me;
- I have not stored passwords, private credentials or confidential data in the repository;
- I understand that if I cannot demonstrate understanding OR provide enough evidence, my project could not be accepted and therefore I will not pass the FISA (Final Integrated Supervised Assessment).

Learner name: _Onkarabetse Maluleka_____

Learner signature: ____o.maluleka_____

Date: _____

Assessor name: _____

Assessor signature: _____

Date: _____

13.1 Source Alignment

This project was developed with reference to the QCTO JavaScript Programmer Skills Programme Document (SP-220373) and the JavaScript Programmer Curriculum Document (Curriculum Code 900219-000-00-00). The detailed monthly rubrics contextualise the programme outcomes and practical skills for an integrated two-month application project.